Afficher ou renvoyer ?

Première NSI

Activité à distance — prévoir, exécuter et réutiliser le résultat d'une fonction

Langages et programmation

Dans le coffre-fort Python du 2 octobre, la fonction `tarif(age)` devait donner un prix. Mais comment faire si un autre calcul a besoin de ce prix ? Une valeur affichée à l'écran ne suffit pas : le programme doit pouvoir la récupérer.

Objectifs

- Distinguer ce qui **s'affiche** de la valeur **renvoyée** par une fonction.
- Écrire une fonction de calcul avec `return`, puis utiliser son résultat ailleurs.
- Décrire simplement les valeurs qu'une fonction accepte et celles qu'elle renvoie.

Prérequis

- Savoir définir et appeler une fonction simple avec `def`.
- Savoir affecter une valeur à une variable.
- Un crayon suffit pour la version papier ; la page HTML interactive demande une connexion Internet au premier chargement de Python.

Sommaire

Étape	Travail
1	Prévoir ce qui s'affiche et ce qui est récupéré
2	Réparer un calcul de prix pour pouvoir le réutiliser
3	Composer deux fonctions et expliquer son choix

> **Repère**
>
> Travail personnel, sans IA. **Prévoyez d'abord** les réponses sur papier, puis utilisez **Exécuter** pour voir les affichages et **Tester** pour vérifier les valeurs renvoyées. Python fonctionne directement dans la page HTML : rien à installer. Si vous n'avez pas de connexion, faites l'activité sur papier avec le PDF. Gardez vos réponses pour la reprise en classe ; consultez les corrigés seulement après votre tentative.

1. Même nombre à l'écran, résultat différent

Voici deux fonctions. Lisez-les **sans cliquer tout de suite** sur les boutons.

```
def double_affiche(n):
    print(2 * n)

def double(n):
    return 2 * n

a = double_affiche(4)
b = double(4)
print("a =", a)
print("b =", b)
```

```
double(4) == 8
double_affiche(4) == 8
```

Le programme exécute les lignes **dans cet ordre**. Le deuxième test échoue au départ : c'est normal, nous allons comprendre pourquoi.

1. Que voit-on à l'écran après l'appel de `double_affiche(4)` ? Et après celui de `double(4)` ?
2. Complétez, sans exécuter : `a = …` et `b = …`.
3. Écrivez les trois lignes qui apparaissent finalement à l'écran, dans l'ordre.
4. Parmi `a + 3` et `b + 3`, quelle expression permet d'obtenir `11` ? Pourquoi ?

Cliquez sur **Exécuter** pour confronter vos prévisions aux trois lignes affichées, puis sur **Tester**. Dans la ligne qui échoue, que lit-on dans la colonne « Obtenu » ? Remplacez ensuite, dans `double_affiche`, `print` par `return` et relancez les tests. Quelle valeur prend maintenant `a` ?

À retenir

`print` montre une valeur à une personne. `return` remet une valeur au code qui a appelé la fonction. Sans `return` exécuté, Python renvoie `None` : ce n'est pas un nombre utilisable pour la suite d'un calcul.

2. Réparer la billetterie

Le coffre-fort du 2 octobre utilisait déjà un tarif de musée. Ici, un élève veut calculer le prix de plusieurs billets identiques, puis ajouter 2 € de frais de réservation.

```
def prix_billets(prix_unitaire, quantite):
    print(prix_unitaire * quantite)

sous_total = prix_billets(7, 3)
total = sous_total + 2
print("Total :", total, "€")
```

1. Quel montant la fonction affiche-t-elle ? Quelle valeur est placée dans `sous_total` ? La ligne `total = sous_total + 2` peut-elle fonctionner ?
2. Cliquez sur **Tester** dans l'éditeur ci-dessous, puis réécrivez **seulement la fonction** `prix_billets` pour que les trois tests passent. Laissez l'affichage final **en dehors** de cette fonction.

   ```
   def prix_billets(prix_unitaire, quantite):
       print(prix_unitaire * quantite)
   ```

   ```
   prix_billets(7, 3) == 21
   prix_billets(5, 2) == 10
   prix_billets(10, 0) == 0
   ```

3. Avec votre version, complétez :

Appel	Valeur renvoyée
`prix_billets(7, 3)`	…
`prix_billets(5, 2)`	…
`prix_billets(10, 0)`	…

4. Après correction, qu'affiche la dernière ligne du programme ?

Dans la page HTML, vous pouvez aussi ajouter **après la fonction** les trois lignes du programme de billetterie pour le vérifier avec **Exécuter**.

> **Repère**
>
> En mathématiques, une fonction prend une valeur dans un **ensemble de départ** et donne une valeur dans un **ensemble d'arrivée**. Ici, la valeur de départ est un couple de nombres entiers (`prix_unitaire` et `quantite`) ; la fonction renvoie un nombre entier (un prix en euros). En Python, on peut dire : `prix_billets : (int, int) → int` . Cette écriture décrit les **types** des valeurs attendues et du résultat ; elle ne remplace pas le calcul.

3. Une fonction peut en appeler une autre

On retrouve la fonction `tarif(age)` du coffre-fort : elle **renvoie** 5 € si l'âge est inférieur à 12 ans, 7 € à partir de 65 ans, et 10 € dans les autres cas. Vous n'avez pas besoin de la réécrire ici.

Écrivez `prix_pour_deux(age1, age2)` qui **renvoie** le prix total pour deux personnes en utilisant **deux appels** à `tarif` . Elle n'affiche rien. Dans l'éditeur HTML, `tarif` est déjà fourni : complétez seulement `prix_pour_deux` , puis cliquez sur **Tester**.

```
def tarif(age):
    if age < 12:
        return 5
    if age >= 65:
        return 7
    return 10

def prix_pour_deux(age1, age2):
    # À compléter
    ...
```

```
prix_pour_deux(10, 70) == 12
prix_pour_deux(14, 20) == 20
prix_pour_deux(11, 12) == 15
```

Vérifiez votre idée sur papier : `prix_pour_deux(10, 70)` doit renvoyer `12` et `prix_pour_deux(14, 20)` doit renvoyer `20` .

Complétez ensuite la phrase : « `prix_pour_deux` prend deux valeurs de type … dans son ensemble de départ et renvoie une valeur de type … dans son ensemble d'arrivée. »

Enfin, écrivez **une ligne à l'extérieur** de la fonction pour afficher le prix de deux personnes de 10 et 70 ans sous la forme `Prix : 12 €` .

Pour conclure

Un camarade propose de remplacer le `return` de `prix_pour_deux` par un `print` : « On verrait le bon prix, donc ça marcherait. » Répondez-lui en deux phrases : dans quel cas a-t-il raison, et qu'est-ce qu'on ne pourrait plus faire facilement ?

Corrigés

Première NSI

Activité à distance — prévoir, exécuter et réutiliser le résultat d'une fonction

Langages et programmation

1. Même nombre à l'écran, résultat différent

`double_affiche(4)` affiche `8`, mais ne renvoie pas de résultat utile : `a` vaut `None`. `double(4)` renvoie `8`, sans rien afficher au moment de son appel : `b` vaut `8`. Les lignes visibles sont, dans l'ordre :

```
8
a = None
b = 8
```

`b + 3` vaut `11`. `a + 3` produit une erreur, car `None` n'est pas un nombre. Le deuxième test obtient donc `None` au lieu de `8`. Après avoir remplacé `print` par `return` dans `double_affiche`, `a` vaut `8` lors d'une nouvelle exécution.

2. Réparer la billetterie

La première version affiche `21`, mais `sous_total` vaut `None` : l'addition avec `2` échoue. Il faut renvoyer le résultat :

```
def prix_billets(prix_unitaire, quantite):
    return prix_unitaire * quantite
```

Les appels renvoient respectivement `21`, `10` et `0`. La dernière ligne affiche `Total : 23 €`.

3. Une fonction peut en appeler une autre

```
def prix_pour_deux(age1, age2):
    return tarif(age1) + tarif(age2)

print("Prix :", prix_pour_deux(10, 70), "€")
```

Les deux paramètres sont de type `int` (des âges entiers) ; le résultat est de type `int` (un prix entier). Afficher le prix peut convenir si l'on veut seulement le montrer à l'écran. Mais si une autre fonction doit ajouter des frais, comparer deux prix ou calculer une réduction, il lui faut la valeur **renvoyée**.